DOSSIER DE INGENIERÍA · AUDITORÍA FORENSE + REMEDIACIÓN FULLSTACK

ATLAS GYM: DESCUBRIMIENTO Y REMEDIACIÓN INTEGRAL

Diagnóstico Red-Team, Arquitectura Transaccional ANTES vs. AHORA, Decisiones Técnicas y Puesta en Producción

ESTADO DE ENTREGA
PRODUCCIÓN READY
34/34 JAVA · 134/134 VITEST · BUILD OK

STACK BACKEND (RAMALUCAS)	STACK FRONTEND (RAMALUCAS)	TOPOLOGÍA REAL DEL CLIENTE	VEREDICTO TÉCNICO ACTUAL
Spring Boot 3.4.3 · Java 17 · Flyway · MySQL 8	**Next.js 16.3 · React 19 · TS 5.7 · Vitest 5**	**2 Dueños (Hermanos) + 1 Tablet Tótem**	**100% Remediado y Verificado**

Contexto de Partida (Por qué NO hizo falta empezar de cero, pero SÍ operar a corazón abierto): Al someter ambos repositorios a una auditoría escéptica de nivel *Red-Team* previo a la entrega a un cliente real, se comprobó que el modelo de dominio, la estética de interfaz y la cobertura funcional base eran sólidos, pero el sistema escondía **8 fallas estructurales críticas** en la capa de sincronización, cálculo contable, consultas JPA, seguridad de endpoints y despliegue. Con las reglas de negocio confirmadas con el cliente (solo dos hermanos operan la administración sin empleados externos, ventana de pago del 1 al 10 y alerta en Asistencias por exceso de días semanales del plan), se ejecutó una **reingeniería quirúrgica en 5 bloques** sin descartar el trabajo previo.

1. Fase de Descubrimiento: Las 8 Bombas de Tiempo Detectadas en la Auditoría

Antes de escribir una sola línea de código correctivo, se trazó la radiografía forense extremo a extremo entre `atlasgym_front` y `atlasgym_back`. A continuación se resumen los hallazgos exactos que habrían provocado pérdida de datos o inconsistencias contables en la primera semana de operación real:

#	Hallazgo Crítico	Evidencia Técnica en Código Original (ANTES)	Impacto Real en el Gimnasio
H1	**Esquizofrenia en Estados de Cuota y Sin Multi-Mes**	`AlumnoService.calcularEstadoCuenta()`, `PagoService` y `lib/date-utils.ts` usaban umbrales distintos. Además, `Pago` no tenía `cantidad_meses`: todo pago cubría solo 1 mes (`periodo_mes`).	Un socio figuraba `AL_DIA` en una vista y `PENDIENTE`/`MOROSO` en otra. Si pagaba una promo trimestral o semestral, al mes siguiente el sistema lo marcaba deudor.
H2	**Tormenta de Queries N+1 en JPA/Hibernate**	`AlumnoService.listarTodos()` hacía `alumnoRepository.findAll()` y por cada socio disparaba queries lazy a `usuario`, `plan`, `rutina` y `pagoRepository.findByAlumnoId ...`.	Con 500 socios activos, un solo `GET /api/alumnos` ejecutaba **más de 1.000 consultas SQL** seriadas contra MySQL, degradando el dashboard y el celular de los dueños.
H3	**Cupo Semanal Decorativo y Tótem Local-First Roto**	`Plan.diaPorSemana` (ej. `"3 días"`) era texto muerto no evaluado al fichar. En `app/totem/page.tsx` se llamaba primero al store local de la tablet y recién después a la API, con rate-limit de `30 req/min`.	Socios con plan de 2 o 3 días entrenaban 6 días sin ninguna alerta en recepción. En hora pico, el ingreso #31 recibía HTTP `429` y veía falsamente *"No identificamos tu DNI"*.
H4	**"Doble Verdad", Mocks Zombies y `.catch(() ⇒ {})`**	`hidratarDesdeBackend()` hacía `if (dto.length > 0)`: si MySQL devolvía `[]`, conservaba `ALUMNOS_MOCK`. Las mutaciones en `lib/store.tsx` tenían `.catch(() ⇒ {})` silenciosos sin *rollback*.	Si el Hermano 1 cargaba un cobro con token vencido o corte de red, su navegador mostraba "éxito" en `localStorage`, pero MySQL nunca se enteraba y el Hermano 2 veía otra caja.
H5	**Brechas de Seguridad REST (IDOR, XSS, Fuerza Bruta)**	`/api/auth/login` sin límite de intentos; `PUT /api/auth/perfil` cambiaba clave sin pedir `passwordActual` y aceptaba `javascript:` en `fotoUrl`; `GET /api/rutinas/{id}` permitía IDOR entre socios; `DataInitializer` logueaba `admin123`.	Exposición a ataques de diccionario, robo de sesión abierta para cambio de contraseña, lectura cruzada de rutinas personalizadas y credenciales por defecto en producción.
H6	**Esquema Sin Gobierno (`ddl-auto=update`)**	Sin Flyway ni SQL base versionado. Hibernate alteraba tablas en caliente al iniciar sobre la base productiva.	Cualquier refactor de entidad JPA en futuras actualizaciones podía corromper columnas, omitir índices críticos o fallar a mitad de arranque sin control transaccional.
H7	**Desfasaje Horario UTC vs. Argentina (21:00 hs)**	Uso de fechas sin fijar `TZ=America/Argentina/Buenos_Aires` en JVM/MySQL/Docker y riesgos de conversión ISO UTC.	A las 21:00 hs de Argentina (00:00 UTC), el servidor saltaba al día siguiente: el día 10 a las 21:15 hs marcaba a todos `PENDIENTE` antes de tiempo y fechaba mal las asistencias nocturnas.

#	Hallazgo Crítico	Evidencia Técnica en Código Original (ANTES)	Impacto Real en el Gimnasio
H8	**Sin Empaquetado Docker ni Backups Consistentes**	Ausencia de `Dockerfile`, `docker-compose.yml`, `.env.example`, exclusión de `.env` en `.gitignore`, regla LF en `.gitattributes` y scripts de `mysqldump`.	Si fallaba el disco de la PC/VPS del gimnasio, se perdía toda la contabilidad histórica, fichas de socios y pagos sin respaldo recuperable.

2. Bloque 1: Máquina de Estados de Cuotas (Opción A), Multi-Mes y Fin del N+1

Descubrimiento Técnico Detallado: En el diseño original, cada registro en la tabla `pagos` representaba únicamente un string `periodo_mes` (formato `YYYY-MM`). No existía aritmética de cobertura plurimensual. Peor aún, para calcular el campo derivado `estadoCuenta` de cada socio al listar el padrón, el backend ejecutaba consultas individuales perezosas (*Lazy Loading*) por cada fila de `alumnos`, mientras que el frontend recalculaba estados con reglas distintas en `lib/date-utils.ts`.

CÓMO ANDABA ANTES

- Criterios contradictorios entre backend y frontend sobre cuándo un socio pasaba a `PENDIENTE`, `MOROSO` o `INACTIVO`.
- Imposible registrar promociones de 2, 3, 6 o 12 meses adelantados sin cargar múltiples pagos ficticios mes a mes.
- `GET /api/alumnos` disparaba `1 + N` queries a `pagos` + `3N` accesos a relaciones -> **>1.000 queries SQL** con 500 socios.

CÓMO ANDA AHORA

- Máquina de estados determinística 100% isomórfica entre Java (`AlumnoService`) y TypeScript (`date-utils.ts`) validada con tests cruzados.
- Columna `cantidad_meses` nativa: un pago en `2026-10` con `cantidadMeses = 3` cubre automáticamente hasta `2026-12` inclusive.
- Complejidad SQL constante **O(1): exactamente 2 consultas SQL** totales sin importar el tamaño del padrón de socios.

Tecnicismo 1.1 — Álgebra Exacta de `YearMonth` para la Máquina de Estados (Opción A)

ISOMORFISMO JAVA + TS

Se formalizó la función pura de estado sobre el calendario local del gimnasio (`hoy`, `diaDelMes = hoy.getDayOfMonth()`, `mesActual = YearMonth.from(hoy)`). Para cada pago con `estadoPago == PAGADO`, se toma su período base `ymBase` (desde `periodoMes` o `YearMonth.from(fecha)`) y su cobertura `n = Math.max(1, cantidadMeses)`, obteniendo el último mes cubierto `ymFin = ymBase.plusMonths(n - 1)`. Sea `maxCubierto` el máximo `ymFin` del socio y `diffMeses = ChronoUnit.MONTHS.between(maxCubierto, mesActual)`:

Condición Evaluada (Orden de Precedencia)	Fórmula Matemática	Estado Resultante
1. Baja administrativa explícita	`!alumno.isActivo()`	`INACTIVO`
2. Mes actual o meses futuros ya pagados (incluye promos multi-mes)	`diffMeses ≤ 0`	`AL_DIA`
3. Cubierto hasta el mes inmediato anterior, dentro de ventana del 1 al 10	`diffMeses == 1 && diaDelMes ≤ 10`	`AL_DIA`
4. Cubierto hasta el mes inmediato anterior, vencida la ventana (día 11 al 31)	`diffMeses == 1 && diaDelMes ≥ 11`	`PENDIENTE`
5. Dejó pasar todo el mes sin abonar (ya cursando el mes subsiguiente)	`diffMeses == 2`	`MOROSO`
6. Abandono prolongado sin cobertura vigente	`diffMeses ≥ 3 && diasDesdeUltimoMovimiento > 60`	`INACTIVO`
7. Socio recién inscripto (`0 pagos PAGADOS`) según antigüedad desde `fechaAlta`	`mesesAlta ≤ 0` → `PENDIENTE` · `== 1` → `MOROSO` · `≥ 2 o >60d` → `INACTIVO`	Según antigüedad

Tecnicismo 1.2 — Erradicación del N+1 mediante `JOIN FETCH` + Batch Hydration

JPA / HIBERNATE 6

En `AlumnoRepository.java` y `PagoRepository.java` se reemplazó la navegación perezosa fila por fila por dos consultas precargadas y ensambladas en memoria en tiempo lineal `O(N)`:

```
// Query 1 (AlumnoRepository): trae todos los socios + usuario + plan + rutina en un único SELECT con LEFT JOIN FETCH
@Query("SELECT a FROM Alumno a LEFT JOIN FETCH a.usuario LEFT JOIN FETCH a.planAlumno LEFT JOIN FETCH a.rutinaAlumno")
List<Alumno> findAllWithRelations();

// Query 2 (PagoRepository): trae todos los pagos de la colección de IDs en un solo SELECT WHERE alumno_id IN (...)
@Query("SELECT p FROM Pago p LEFT JOIN FETCH p.planPago WHERE p.alumno.idAlumno IN :alumnoIds ORDER BY p.fecha DESC")
List<Pago> findByAlumnoIdsOrderByFechaDesc(@Param("alumnoIds") Collection<Long> alumnoIds);
```

Adicionalmente, en `app/finanzas/page.tsx` se incorporó el selector de duración (`1, 2, 3, 6, 12 meses`) que autocalcula el monto sugerido multiplicando el valor del plan, renderiza badges `×N meses` en la tabla contable, proyecta el mes exacto de vencimiento en `VistaCuotasAlumno` e incluye la columna en la exportación CSV.

3. Bloque 2: Cupo Semanal de Días por Plan, Tótem MySQL-First y Monitor en Vivo

Descubrimiento Técnico Detallado: El cliente pidió explícitamente: "*que el tótem no trabe la puerta, pero que le avise al módulo de asistencia si hay alguien que ya vino los días que le corresponde, igual que avisa si debe cuota*". Al auditar el código encontramos dos problemas graves: (1) ningún servicio parseaba cuántos días semanales permitía el plan ni contaba días distintos asistidos en la semana; (2) `app/totem/page.tsx` ejecutaba primero la mutación sobre `localStorage` de la tablet y ocultaba errores HTTP (como `429 Too Many Requests` al superar 30 ingresos/min o `403 Forbidden`) detrás del mensaje genérico "*No identificamos tu DNI*".

CÓMO ANDABA ANTES

- El campo `diaPorSemana` (ej. `"3 días por semana"`) era meramente informativo: 0 control sobre asistencias semanales.
- Si un socio fichaba en la tablet del Tótem, el celular/notebook de los dueños en `/asistencias` no se enteraba en vivo sin recargar manualmente.
- Rate-limit de 30 consultas/minuto en `AsistenciaController` colapsaba en el cambio de turno de las 18:00–20:00 hs.

CÓMO ANDA AHORA

- Conteo exacto de **días distintos asistidos en la semana ISO (Lunes a Domingo)** con deduplicación automática de doble turno diario.
- Tótem **MySQL-First** con soporte de `120 req/min` , purga de memoria IP, secreto opcional `X-Totem-Secret` y fallback offline limpio.
- `/asistencias` hace *polling* inteligente cada 5 s y despliega banner naranja prominente, tarjetas resaltadas y filtro `Excedidos plan` .

Tecnicismo 2.1 — Conteo Semanal Lunes–Domingo con Deduplicación de Doble Ingreso

BACKEND + FRONTEND

Se implementó el extractor `extraerLimiteDiasSemana(String diaPorSemana)` tanto en `AsistenciaService.java` como en `lib/asistencia-utils.ts` . Si el texto contiene términos libres (`"libre"` , `"ilimitado"` , `"full"` , `"todos"`) retorna `null` (sin tope); si matchea `(\d+)\s*d[ií]as?` extrae el entero `limite` . Para cualquier asistencia en una fecha `F` :

- Se calcula el lunes de esa semana ISO: `inicioSemana = F.with(TemporalAdjusters.previousOrSame(DayOfWeek.MONDAY))` (y en TypeScript con aritmética local `offsetLunes = (fecha.getDay() + 6) % 7` sin pasar por `toISOString()` para jamás desplazar el día por UTC).
- En `AsistenciaRepository.java` se consulta `SELECT DISTINCT a.fecha FROM Asistencia a WHERE a.alumno.idAlumno = :id AND a.fecha BETWEEN :lunes AND :hasta` (y su variante batch `findByAlumnoIdsAndRango` para listados históricos sin N+1).
- Si un socio con plan de **3 días** ficha el lunes a la mañana y vuelve el lunes a la tarde (o tipea dos veces su DNI en el Tótem), ambos registros cuentan como **1 solo día asistido**. Al fichar un 4to día distinto entre lunes y domingo, el DTO expone `asistenciasSemana = 4` , `limiteDiasSemana = 3` , `excedeLimiteSemanal = true` .

Tecnicismo 2.2 — Flujo MySQL-First en `/totem` y Telemetría Reactiva en `/asistencias`

TIEMPO REAL MULTI-DISPOSITIVO

- **Inversión de prioridad en `app/totem/page.tsx`:** Cuando `isApiRuntimeEnabled()` está activo, invoca primero `await apiCheckInTotem(valorDni)` enviando `X-Totem-Secret` (si se configuró `NEXT_PUBLIC_TOTEM_SECRET`). Si el backend responde HTTP `403` o `429`, exhibe el motivo real de seguridad/límite en pantalla en vez de mentir diciendo que el DNI no existe. Solo si `esErrorDeRed(err)` detecta caída física de red, opera en contingencia local con `omitirApiRemota: true`.
- **Proxy & Rate-Limit en `AsistenciaController.java`:** Se activó `server.forward-headers-strategy=framework` para leer la IP real detrás de Nginx/reverse-proxy, se elevó la ventana a `120 req/min` (2 check-ins por segundo sostenidos) y se agregó limpieza automática del `ConcurrentHashMap` para evitar fugas de memoria (*memory leaks*).
- **Monitor vivo en `app/asistencias/page.tsx`:** Efecto de actualización cada `5.000 ms` activo únicamente mientras `document.visibilityState === "visible"` + disparo inmediato al recuperar foco. Cuando ingresa alguien excedido en días, aparece una alerta naranja superior sobre "*Socios en sala ahora*", badge `Excedió plan (4/3 d.)`, filtro rápido `Excedidos plan` y columnas dedicadas en el reporte CSV.

4. Bloque 3: Fin de la "Doble Verdad" Multi-Dispositivo y Hardening de Seguridad

Descubrimiento Técnico Detallado: Como el gimnasio es administrado en simultáneo desde los celulares/notebooks de los dos hermanos dueños más la tablet del Tótem, usar `localStorage` como fuente primaria con llamadas REST *fire-and-forget* (`apiCrearPago( … ).catch(() ⇒ {})`) era catastrófico. Además, cuando la base MySQL recién instalada devolvía una lista vacía `[]`, la condición `if (alumnosDto.length > 0)` fallaba y dejaba vivos en pantalla los socios ficticios de demostración (`ALUMNOS_MOCK`).

CÓMO ANDABA ANTES

- Al vaciar una tabla en MySQL y presionar `F5`, reaparecían mágicamente los datos mockeados de desarrollo.
- 17 operaciones de creación/edición/borrado silenciaban fallos del servidor con `.catch(() ⇒ {})` dejando la UI desfasada de MySQL.
- `ModalPerfil` mostraba "*Contraseña actualizada*" sin esperar respuesta del servidor ni validar la contraseña actual.

CÓMO ANDA AHORA

- MySQL es la única fuente de verdad: arrays vacíos `[]` pisan limpiamente cualquier mock gracias a `atlas_backend_synced_v1`.
- Todas las mutaciones aplican **Snapshot + Rollback Automático** ante error HTTP y notifican con Toast rojo descriptivo en `AppShell`.
- Endurecimiento completo de autenticación, perfil, anti-IDOR, validación Bean Validation y credenciales iniciales parametrizadas.

Tecnicismo 3.1 — Patrón Optimistic Rollback, Anti-Mocks Zombies y Re-Hidratación por Foco

REACT 19 STORE ARCHITECTURE

Se refactorizó íntegramente `lib/store.tsx`, `lib/use-avisos.ts`, `components/app-shell.tsx` y `components/modal-perfil.tsx`:

- **Erradicación de Mocks Zombies:** Al concretar la primera hidratación exitosa desde Spring Boot, se marca `atlas_backend_synced_v1 = "true"` y se evalúa `Array.isArray(dto)` en lugar de `dto.length > 0`. Si MySQL tiene 0 socios, 0 pagos o 0 avisos, el frontend refleja exactamente 0 registros y persiste ese estado limpio.
- **Sincronización Cruzada entre los Dos Hermanos:** Se suscribieron listeners a `window.focus` y `document.visibilitychange` (con *throttle* de 8 segundos) que re-ejecutan `hidratarDesdeBackend()` automáticamente apenas cualquiera de los dueños desbloquea el celular o vuelve a la pestaña.
- **Rollback Transaccional en Cliente:** En las 17 mutaciones del sistema (alumnos, pagos, planes, rutinas, asignación de rutinas, videoteca, sesiones de entrenamiento, asistencias y avisos), se captura el estado previo antes de actualizar la UI; si la promesa HTTP rechaza (400, 401, 403, 404, 500 o timeout), se restaura instantáneamente el snapshot anterior y se emite `notificarErrorSync()` (con detección específica de `HTTP 401` para avisar que expiró la sesión JWT).

Tecnicismo 3.2 — Cinco Controles de Seguridad Cerrados en Backend (`SeguridadHardeningTest`)

SPRING SECURITY + JJWT

Vector Auditado	Mecanismo Implementado en Backend	Prueba Automatizada
Fuerza Bruta en Login `POST /api/auth/login`	`AuthService` rastrea fallos en `ConcurrentHashMap<String, RegistroIntentos>`. Al 5to intento fallido bloquea el identificador por 120 segundos devolviendo HTTP `429 Too Many Requests`.	5 intentos `401` -> 6to intento bloqueado con `429` aun con clave real.
Secuestro de Perfil y XSS `PUT /api/auth/perfil`	Exige `passwordActual` verificada contra hash BCrypt antes de admitir nueva clave (>= 6 chars). Valida `esUrlImagenSegura()` permitiendo solo `https://`, `http://` o `/` (bloquea `javascript:`).	Rechaza con `400` sin clave actual, con clave errónea o con `javascript:alert(1)`.
IDOR en Rutinas Privadas `GET /api/rutinas/{id}`	`RutinaController.buscarPorId` verifica rol: si no es `ROLE_ADMIN` y `!rutina.esGenerica()`, comprueba que el ID coincida con la rutina asignada al alumno autenticado; si no, lanza `AccessDeniedException`.	Alumno A recibe `200` en la suya/genérica y `403 Forbidden` en la de Alumno B.
Inyección de Datos Inválidos `POST /api/videoteca`	`VideoTecnicaDto` anotado con `@NotBlank` en `titulo` y `@Size` exactos (`150`, `50`, `500`, `100`) alineados al DDL de MySQL, evaluados por `@Valid` y `GlobalExceptionHandler`.	Payload con título vacío retorna `400 Bad Request` con detalle de campo.
Credenciales Hardcodeadas `DataInitializer.java`	Parametrizadas vía `atlas.admin.email` y `atlas.admin.password`. Se eliminó el logueo de la contraseña en texto plano y en producción no recrea cuentas si ya existe un `ADMIN` activo.	Validado en arranque con perfil de producción y Flyway activo.

5. Bloque 4: Migraciones Versionadas con Flyway 10 y `ddl-auto=validate`

Descubrimiento Técnico Detallado: El backend operaba con `spring.jpa.hibernate.ddl-auto=update` como configuración base y carecía de motor de migraciones SQL. En producción, dejar que el ORM modifique tablas financieras y de socios automáticamente sin scripts versionados ni índices explícitos es una de las causas más frecuentes de corrupción silenciosa o bloqueos de arranque tras una actualización.

CÓMO ANDABA ANTES

- 0 scripts SQL versionados; el esquema dependía de lo que Hibernate infiriera en tiempo de ejecución.
- Faltaban índices compuestos de lectura intensiva en tablas de crecimiento diario (`asistencias`, `pagos`, `sesiones_entrenamiento`).
- Secretos con valores por defecto débiles embebidos en el único `application.properties`.

CÓMO ANDA AHORA

- `flyway-core` + `flyway-mysql` gobiernan el esquema mediante `V1__esquema_inicial.sql` (16 tablas exactas con integridad referencial).
- Perfil `application-prod.properties` con `ddl-auto=validate`: Hibernate valida cada columna al arrancar sin tocar el DDL.
- Test automatizado `FlywayEsquemaValidacionTest` certifica en cada build que SQL y entidades JPA coinciden al 100%.

Tecnicismo 4.1 — Arquitectura Relacional de las 16 Tablas en `V1__esquema_inicial.sql`

DDL MYSQL 8.0 + ÍNDICES B-TREE

Se relevaron las 13 clases `@Entity` y las 3 tablas de colección/unión (`@ElementCollection` y `@ManyToMany`) para escribir un DDL determinístico con políticas de borrado contablemente seguras:

- **Núcleo de Identidad y Catálogos:** `usuarios` (`uk_usuarios_email`), `planes` (`DECIMAL(10,2)`), `rutinas` , `videos_tecnica` y `video_consejos` (`ON DELETE CASCADE`).
- **Estructura de Entrenamiento:** `dias_rutina` , tabla puente `dias_rutina_videos` y `ejercicios` (`ON DELETE SET NULL` hacia `videos_tecnica` para no romper rutinas si se borra un video).
- **Padrón y Contabilidad Protegida:** `alumnos` (`uk_alumnos_dni` , `uk_alumnos_usuario`) y `pagos` (con `cantidad_meses INT DEFAULT 1` , `alumno_nombre_historico VARCHAR(150)` y `ON DELETE SET NULL` sobre `alumno_id` / `plan_id` para que **jamás se pierda un registro de caja histórico** aunque se elimine un socio o plan).
- **Operación Diaria y Progreso:** `asistencias` , `avisos` , `aviso_leidos` , `sesiones_entrenamiento` , `sesiones_ejercicios` y `registros_series` (`DOUBLE PRECISION` para kilajes fraccionarios).
- **Índices Compuestos de Alta Frecuencia:** `idx_pagos_alumno_fecha (alumno_id, fecha)` , `idx_asistencias_alumno_fecha (alumno_id, fecha)` e `idx_sesiones_alumno_fecha (alumno_id, fecha)` , acelerando el check-in del Tótem y el cálculo de cupos semanales a sub-milisegundos.

6. Bloque 5: Infraestructura Docker, Blindaje Horario UTC-3 y Backups Diarios

Descubrimiento Técnico Detallado: Durante el diseño del contenedor productivo detectamos dos trampas clásicas de despliegue real en Argentina: (1) Las imágenes Linux Alpine/Debian corren por defecto en `UTC` ; a las **21:00 hs de Buenos Aires** ya es las **00:00 hs del día siguiente** en el contenedor, lo que adelantaba 3 horas el vencimiento del día 10 y registraba mal la fecha de quienes entrenan de noche. (2) Clonar scripts `.sh` desde Windows con `autocrlf` inyecta `\r\n` y rompe `cron` en Linux con `/usr/bin/env: 'bash\r'` .

CÓMO ANDABA ANTES

- Despliegue manual dependiente de tener JDK 17, Maven, Node 20 y MySQL instalados a mano en la máquina destino.
- `.gitignore` del backend no bloqueaba archivos `.env` ni carpetas `backups/` , con riesgo de filtrar claves a GitHub.
- Sin estrategia automatizada de respaldo ni retención de base de datos.

CÓMO ANDA AHORA

- Orquestación completa en un comando (`docker compose up -d --build`) con imágenes multi-stage livianas y usuarios no-root.
- Reloj sincronizado a `America/Argentina/Buenos_Aires` (`-03:00`) en los 3 contenedores (MySQL + JVM + Node.js).
- Scripts `backup-mysql.sh` (LF garantizado) y `backup-mysql.ps1` con snapshot transaccional `.sql.gz` y rotación a 14 días.

Tecnicismo 5.1 — Contenedores Multi-Stage Blindados y Aislamiento de Red

DOCKER + COMPOSE PROD

- **Backend (`atlasgym_back/Dockerfile`):** Compila en etapa efímera `maven:3.9-eclipse-temurin-17-alpine` y copia únicamente el JAR final a `eclipse-temurin:17-jre-alpine` con `tzdata` , usuario sin privilegios `atlas` y flags JVM `-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -Duser.timezone=America/Argentina/Buenos_Aires` .
- **Frontend (`atlasgym_front/Dockerfile` + `next.config.mjs`):** Activa condicionalmente `output: process.env.DOCKER_BUILD === "1" ? "standalone" : undefined` (evitando conflictos de symlinks en Windows local pero generando el bundle mínimo `server.js` en Docker sobre `node:20-alpine` con usuario `nextjs` UID 1001).
- **Orquestación (`docker-compose.yml`):** MySQL 8.0 configurado con `utf8mb4_unicode_ci` , `--default-time-zone=-03:00` , *healthcheck* activo antes de iniciar Spring Boot y puerto bindeado exclusivamente a loopback local (`127.0.0.1:${DB_HOST_PORT:-3307}:3306`) para que la base de datos sea invisible desde Internet público. Exige mediante sintaxis `${VAR:?mensaje}` que todos los secretos productivos estén definidos en `.env` .
- **Backups Consistentes en Caliente:** `scripts/backup-mysql.sh` y `scripts/backup-mysql.ps1` ejecutan `mysqldump --single-transaction --quick --routines --triggers` (lee un snapshot MVCC consistente de InnoDB sin bloquear el Tótem ni la caja), comprimen en `gzip -9` sobre un archivo temporal `.tmp` , verifican tamaño no nulo antes de confirmar, restringen permisos con `chmod 600` y eliminan respaldos de más de 14 días.

7. Evidencia de Verificación Automatizada e Inventario de Archivos

34 / 34	134 / 134	0 Errores	13 / 13
TESTS BACKEND JUNIT 5 + MOCKMVC + FLYWAY	TESTS FRONTEND VITEST (15 SUITES)	TYPESCRIPT STRICT (TSC --NOEMIT)	RUTAS COMPILADAS (NEXT BUILD PROD)

7.1. Nuevas Suites de Pruebas de Regresión Creadas

Archivo de Prueba	Entorno	Casos Críticos Certificados
`CuotasEstadoYMultiMesTest.java`	Spring Boot / H2	Transiciones exactas día 10 (`AL_DIA`) -> día 11 (`PENDIENTE`) -> mes siguiente (`MOROSO`) -> abandono >60d (`INACTIVO`), cobertura de pagos trimestrales/semestrales (`cantidadMeses`) y socio nuevo con 0 pagos.
`TotemYLimiteSemanalTest.java`	Spring Boot / MockMvc	Parser de planes con cupo vs libres, deduplicación de doble fichaje el mismo día, alerta `excedeLimiteSemanal=true` al 4to día de un plan de 3 días sin bloquear el ingreso y ráfaga de hora pico en `/api/asistencias/totem` sin disparar HTTP 429.
`SeguridadHardeningTest.java`	Spring Boot / Security	Bloqueo HTTP 429 al 6to login fallido, rechazo HTTP 400 en cambio de clave sin `passwordActual` válida o `fotoUrl` con `javascript:` , bloqueo IDOR HTTP 403 entre socios en rutinas privadas, validación `@Valid` en videoteca y validación de `X-Totem-Secret` .
`FlywayEsquemaValidacionTest.java`	Flyway 10 + Hibernate 6	Levanta base limpia ejecutando `V1__esquema_inicial.sql` con `spring.jpa.hibernate.ddl-auto=validate` y `atlas.seed.enabled=false` , certificando 0 discrepancias entre las 16 tablas SQL y las entidades JPA.
`lib/cuotas-estados.test.ts` `lib/asistencias.test.ts` `lib/api-mappers.test.ts`	Vitest 5 / Frontend	Isomorfismo 1:1 con Java en cálculo de estados de cuota `Opción A` , promos multi-mes, conteo semanal Lunes–Domingo sin desviación UTC, deduplicación de doble turno y mapeo de DTOs del Tótem y asistencias.

7.2. Mapa Completo de Archivos Intervenidos por Repositorio

Repositorio / Capa	Archivos Modificados y Creados
Backend — Dominio, DTOs y Repositorios (`atlasgym_back`)	`Modelo/Pago.java` · `Dto/PagoRequestDto.java` · `Dto/PagoResponseDto.java` · `Dto/AsistenciaResponseDto.java` · `Dto/TotemCheckInResponseDto.java` · `Dto/VideoTecnicaDto.java` · `Repositorio/AlumnoRepository.java` · `Repositorio/PagoRepository.java` · `Repositorio/AsistenciaRepository.java`
Backend — Servicios, Seguridad y Control (`atlasgym_back`)	`Servicio/AlumnoService.java` · `Servicio/PagoService.java` · `Servicio/AsistenciaService.java` · `Servicio/AuthService.java` · `Controlador/AsistenciaController.java` · `Controlador/RutinaController.java` · `Controlador/VideoTecnicaController.java` · `Seguridad/SecurityConfig.java` · `Excepciones/GlobalExceptionHandler.java` · `Config/DataInitializer.java`
Backend — Flyway, Config y DevOps (`atlasgym_back`)	`pom.xml` · `src/main/resources/db/migration/V1__esquema_inicial.sql` · `application.properties` · `application-prod.properties` · `src/test/resources/application.properties` · `Dockerfile` · `.dockerignore` · `docker-compose.yml` · `.env.example` · `scripts/backup-mysql.sh` · `scripts/backup-mysql.ps1` · `.gitignore` · `.gitattributes`
Frontend — Núcleo, Sync y Utilidades (`atlasgym_front`)	`lib/types.ts` · `lib/date-utils.ts` · `lib/asistencia-utils.ts` · `lib/api-client.ts` · `lib/api-mappers.ts` · `lib/store.tsx` · `lib/use-avisos.ts`
Frontend — Módulos UI y Despliegue (`atlasgym_front`)	`app/finanzas/page.tsx` · `app/totem/page.tsx` · `app/asistencias/page.tsx` · `app/planes/page.tsx` · `components/app-shell.tsx` · `components/modal-perfil.tsx` · `next.config.mjs` · `Dockerfile` · `.dockerignore` · `.env.example` · `.gitignore`

Protocolo Rápido de Puesta en Marcha para Entrega al Cliente: 1) Copiar `atlasgym_back/.env.example` a `atlasgym_back/.env` y definir contraseñas reales + `ATLAS_JWT_SECRET` de 256 bits (`openssl rand -hex 32`) + `ATLAS_TOTEM_SECRET` . 2) Ejecutar `docker compose up -d --build` : MySQL 8 levantará blindado en `127.0.0.1` , Flyway creará y validará las 16 tablas automáticamente con `V1__esquema_inicial.sql` , y se creará únicamente la cuenta administradora de los dueños sin datos basura. 3) Registrar `scripts/backup-mysql.sh` en `cron` (Linux) o `scripts/backup-mysql.ps1` en el Programador de Tareas (Windows) a las 03:30 AM.